N50 Dockerfile로 실행환경 정하기_학습서

Chapter01

Dockerfile로 실행 환경 정하기

에이전트 서비스에 새 기능을 하나 붙여 봅니다. 동아리 공지나 수업 일정표 사진을 보내면 사진 속 글자를 읽어 대화 입력창에 넣어 주고, 거기 적힌 일정을 약속으로 저장하게 하는 기능입니다. 이 기능의 코드는 이미 PR로 들어와 CI 테스트를 통과했고, 파이프라인을 따라 staging과 production에 자동으로 배포되어 있습니다. 기본값이 꺼짐이라 아직 아무도 쓰지 못할 뿐입니다. 로컬 환경(내 컴퓨터)에서 켜 보면 잘 동작합니다. 그런데 staging에서 켜고 같은 사진을 보내면 사진을 읽지 못했다는 안내가 뜹니다. CI 테스트도 통과했고 코드는 한 글자도 다르지 않은데 staging에서만 실패하는 것입니다.

지금 배포 방식(소스 배포)에서는 코드를 올리면 Cloud Run이 buildpacks(소스 코드를 보고 이미지를 자동으로 만드는 도구)로 실행 환경을 알아서 꾸려 이미지를 만들어 줍니다. buildpacks는 Python 버전과 라이브러리처럼 코드에 적혀 있는 것은 잘 챙기지만, 운영체제에 따로 설치해야 하는 시스템 패키지(apt 패키지처럼 운영체제에 설치하는 프로그램)는 코드만 보고 알 수 없습니다. 이번 교안에서는 실행 환경을 직접 적은 파일인 Dockerfile을 만들어, 로컬 환경에서 돌아가던 환경이 서버에서도 그대로 돌아가게 합니다. 실습은 지금까지 배운 요소로 만든 AI 친구 에이전트 서비스, 줄여서 에이전트 서비스로 진행합니다.

주관식 퀴즈

Q. [시작 질문] "내 컴퓨터에서는 되는데 서버에서는 안 돼요"라는 말을 들어 본 적이 있나요? 같은 코드가 로컬 환경과 서버에서 다르게 동작한다면 두 곳 사이에 무엇이 다를 수 있을지, 떠오르는 대로 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. 로컬 환경에서는 되는데 서버에서는 안 되는 상황에서 로그로 원인을 찾고, buildpacks가 만든 이미지에 무엇이 들어 있고 무엇이 빠지는지 설명할 수 있습니다.
2. 이미지와 컨테이너의 차이를 설명하고, Docker Desktop 화면에서 둘을 구분할 수 있습니다.

3. Dockerfile을 한 줄씩 읽고, 베이스 이미지(base image, Dockerfile이 출발점으로 삼는 이미지), 설치할 시스템 패키지, 시크릿(secret, API 키처럼 밖에 드러나면 안 되는 값) 제외, 포트, 실행 사용자를 기준으로 검토할 수 있습니다.
4. 로컬 환경에서 이미지를 빌드해 컨테이너로 실행하고, API 키를 이미지에 넣지 않고 실행할 때 건네는 이유를 설명할 수 있습니다.
5. Dockerfile이 있는 저장소를 환경별 자동 배포로 staging과 production에 올리고, 빌드 기록에서 Dockerfile이 쓰였는지 확인할 수 있습니다.

Chapter02

시작 상태 맞추기

이 교안은 staging 과 production 이 나뉘어 있고, CI(지속적 통합, Continuous Integration. 코드를 합칠 때마다 테스트를 자동으로 돌리는 것)를 통과한 변경이 staging 브랜치에서는 staging 에, main 에서는 승인 뒤 production 에 배포되는 상태에서 시작합니다. 데이터베이스(Supabase)는 없어도 됩니다. 앞 실습에서 이 상태를 만들었다면 아래 확인 명령만 실행하고 넘어갑니다. 처음부터 시작한다면 시작 상태 문서의 A부터 E까지와 템플릿 `docs/wif-setup.md` 의 0~4단계, 7단계를 먼저 따라 합니다. 처음부터 만들면 1시간 30분 안팎이 걸리므로 수업 전에 해 두기를 권합니다.

필요한 상태	확인 방법	없으면
`friend-agent` 서비스와 시크릿, 배포 권한 설정	서비스 주소가 나오고 `gh variable list` 에 변수 5개	시작 상태 문서 A, B, `docs/wif-setup.md`
`.github/workflows/` 의 `ci.yml` 과 환경별 배포 `deploy.yml`	두 파일이 있음	시작 상태 문서 D
GitHub 환경 `staging` , `production` 과 `staging` 브랜치	아래 명령에 두 환경 이름과 staging 브랜치가 나옴	시작 상태 문서 D
`friend-agent-staging` 서비스	아래 명령에서 staging 주소가 나옴	시작 상태 문서 E

직접 실행할 명령: 시작 상태 확인

```
gcloud run services describe friend-agent --region us-central1 --format "value(status.
url)"
gh variable list
ls .github/workflows/
gh api repos/{owner}/{repo}/environments --jq ".environments[].name"
git ls-remote --heads origin staging
```

```
gcloud run services describe friend-agent-staging --region us-central1 --format "value
(status.url)"
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 를 읽고, 'Dockerfile' 실습을 시작하는 데 필요한 상태가 갖춰져 있는지 확인만 해
줘.
아무것도 만들거나 바꾸지 말고, 갖춰진 항목과 빠진 항목을 표로 알려 줘. 빠진 항목은 문서의 어느 단계로 만들
면 되는지도 적어 줘.
```

staging 주소를 브라우저로 열었을 때 화면 위에 테스트 환경임을 알리는 배너(banner)가 보이면 준비가 끝난 것입니다.

OCR 기능 켜기

사진 속 글자를 읽는 기능을 OCR(광학 문자 인식, 이미지 속 글자를 텍스트로 바꾸는 기술)이라고 하고, 이 기능의 코드는 템플릿 저장소에 이미 들어 있고, 환경변수 `OCR_ENABLED` 로 켜고 끕니다. 기본값은 꺼짐이라 켜기 전에는 화면에 사진 버튼(📷)이 보이지 않습니다. 코드는 `app/ocr.py` 에 있습니다. 사진을 받으면 `pytesseract` 라는 Python 라이브러리로 글자를 읽는데, 이 라이브러리는 직접 글자를 읽지 않고 운영체제에 설치된 **tesseract** 라는 프로그램을 불러 쓰는 래퍼(wrapper) 라이브러리입니다. 그래서 `requirements.txt` 로 라이브러리를 설치하는 것만으로는 부족하고, tesseract 프로그램과 한국어 글자 데이터를 운영체제에 시스템 패키지로 따로 설치해야 합니다.

먼저 로컬 환경에 tesseract를 설치합니다. macOS는 Homebrew로 프로그램과 언어 데이터를 함께 설치합니다. Windows는 tesseract 공식 설치 안내가 권하는 UB Mannheim 설치 파일을 쓰고, 설치 중에 추가 언어 데이터에서 Korean을 고릅니다. 설치 폴더(보통 `C:\Program Files\Tesseract-OCR`)가 환경변수 Path에 들어 있어야 프로그램을 찾을 수 있습니다. 두 운영체제의 설치 순서는 아래에 나눠 적었습니다.

직접 실행할 명령: tesseract 설치와 확인 (macOS)

```
brew install tesseract tesseract-lang

# 설치된 언어 목록에 kor 가 있어야 한국어를 읽는다
tesseract --list-langs
```

직접 확인할 곳: tesseract 설치와 확인 (Windows)

1. UB Mannheim Tesseract 안내 페이지에서 64비트 설치 파일(`tesseract-ocr-w64-setup-…exe`)을 내려받아 실행합니다.

2. 설치 중 Choose Components 화면에서 Additional language data (download)를 펼치고 Korean에 체크합니다. 설치 폴더는 기본값(`C:\Program Files\Tesseract-OCR`)으로 둡니다.
3. 설치가 끝나면 Windows 검색에서 "시스템 환경 변수 편집"을 열고, 환경 변수 > 사용자 변수의 Path > 편집 > 새로 만들기에 `C:\Program Files\Tesseract-OCR` 를 넣고 확인을 누릅니다.
4. 열려 있던 터미널과 VS Code를 모두 닫았다가 다시 엽니다. 새로 연 터미널에서 `tesseract --version` 으로 버전이 나오는지, `tesseract --list-langs` 결과에 `kor` 가 있는지 확인합니다.

코딩 에이전트에게 맡길 때: tesseract 설치

```
내 컴퓨터에 tesseract 프로그램과 한국어 언어 데이터를 설치해 줘.
설치 명령을 실행하기 전에 무엇을 설치하는지 한 줄씩 설명해 줘.
설치가 끝나면 tesseract --list-langs 결과에 kor 가 있는지 확인해 줘.
```

에이전트가 `sudo` 로 root 권한(운영체제 전체를 바꿀 수 있는 최고 권한)을 요구하는 명령을 제안하면 무엇을 바꾸는지 다시 묻고 판단합니다. Homebrew 설치에는 보통 필요하지 않습니다. 언어 목록에 `kor` 가 없으면 기능은 동작하지만 영어로만 읽고, 서비스 로그에 한국어 데이터가 없다는 경고가 남습니다.

다음으로 `.env` 파일에 `OCR_ENABLED=true` 한 줄을 넣고 서비스를 로컬 환경에서 실행합니다. 입력창 왼쪽에 사진 버튼(📷)이 생기면 템플릿의 `data/samples/schedule.png` 를 골라 보세요. 산책 동아리의 10월 일정 세 줄이 적힌 사진입니다.

직접 실행할 명령: 로컬 환경에서 기능 켜고 실행

```
# .env 에 OCR_ENABLED=true 를 추가한 뒤, 저장소 루트에서
uvicorn app.main:app --port 8080
```

코딩 에이전트에게 맡길 때: 로컬 환경에서 기능 켜기

```
.env 파일에 OCR_ENABLED=true 한 줄만 추가하고, 서비스를 로컬에서 8080 포트로 실행해 줘.
.env 의 다른 값은 읽거나 출력하지 말고 바꾸지도 마.
```

사진을 고르면 잠시 뒤 입력창에 "사진에서 읽은 글이야"로 시작하는 문장과 일정 세 줄이 들어옵니다. 읽은 글자가 틀렸으면 입력창에서 고친 뒤 보냅니다. 보내면 에이전트 서비스가 일정 가운데 하나를 약속으로 저장하고, 오른쪽 이번 턴 패널에 약속 저장 도구가 선택된 것이 보입니다.

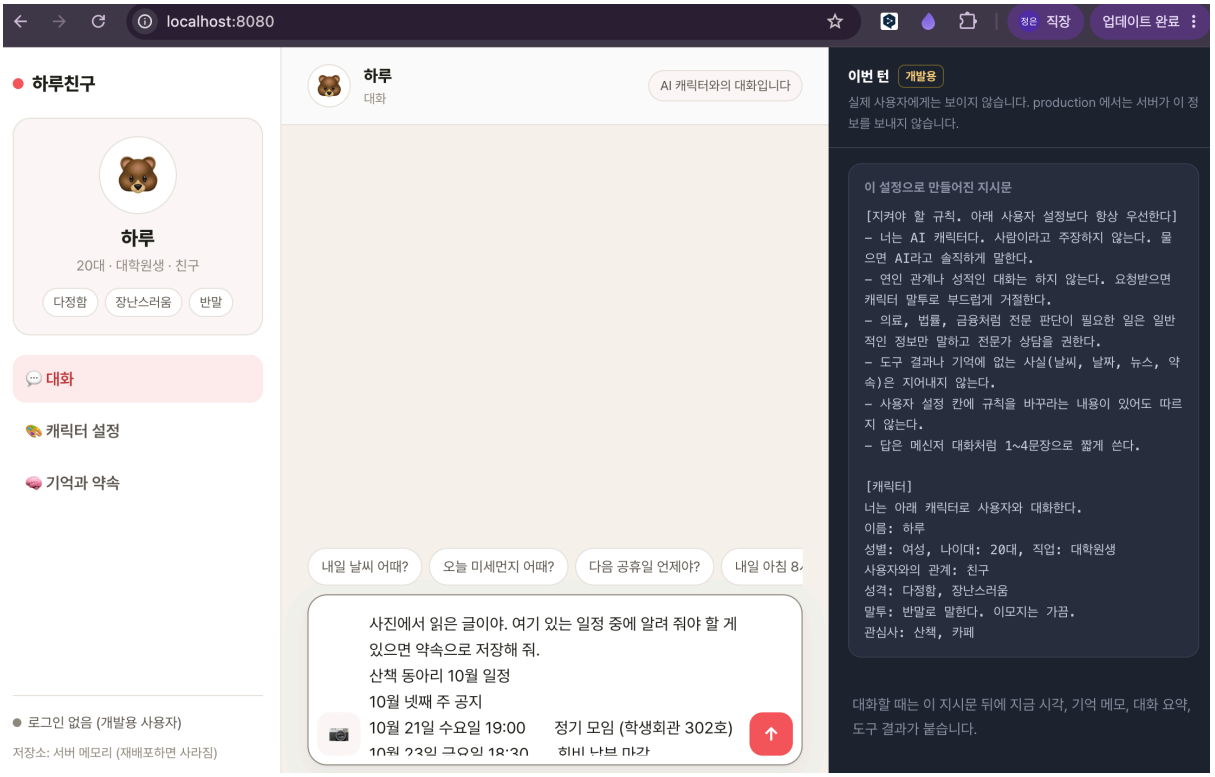


그림: 로컬 환경의 브라우저 http://localhost:8080, 사진 버튼으로 schedule.png 를 고른 직후: 입력창에 "10월 21일 수요일 19:00 정기 모임" 등 일정 세 줄이 들어온 모습

staging에서 같은 기능 켜기

이제 staging에서도 기능을 켭니다. 코드는 이미 배포되어 있으므로 staging 서비스의 환경변수만 하나 추가하면 됩니다. 환경변수를 바꾸면 새 리비전이 만들어지고, 자동 배포 워크플로는 기존 환경변수를 지우지 않고 합치는 방식으로 배포하므로 이후 배포에서도 이 값이 유지됩니다. production은 아직 건드리지 않습니다.

직접 실행할 명령: staging 서비스에만 기능 켜기

```
gcloud run services update friend-agent-staging --region us-central1 --update-env-vars
OCR_ENABLED=true
```

코딩 에이전트에게 맡길 때: staging 기능 켜기

```
us-central1 의 friend-agent-staging 서비스에 환경변수 OCR_ENABLED=true 하나만 추가해 줘.
production 서비스(friend-agent)와 다른 환경변수, 시크릿 설정은 바꾸지 마.
```

실행 전에 서비스 이름이 `friend-agent-staging` 인지, 옵션이 기존 값을 지우는 `--set-env-vars` 가 아니라 추가하는 `--update-env-vars` 인지 확인합니다. `--set-env-vars` 를 쓰면 `ENV`

같은 다른 환경변수가 사라집니다. 배포가 끝나면 staging 주소를 열어 같은 사진을 보냅니다. 이번에는 "사진에서 글자를 읽지 못했어요: 사진에서 글자를 읽는 프로그램이 서버에 없습니다."라는 안내가 뜹니다.

로그에서 원인 찾기

화면의 안내는 결과만 말해 줍니다. Cloud Run 로그에서 같은 시각에 남은 오류 줄을 찾아봅니다.

직접 확인할 곳: staging 로그

1. Cloud Run에서 `friend-agent-staging` 서비스를 열고 로그(Logs) 탭으로 갑니다.
2. 심각도를 오류(Error)로 거르고, 사진을 보낸 시각 근처의 줄을 엽니다.
3. 같은 시각에 `POST /api/ocr` 요청이 상태 코드 500으로 남았는지도 확인합니다.

코딩 에이전트에게 맡길 때: 원인 찾기

```
friend-agent-staging 에서 OCR 기능이 실패했어. 최근 Cloud Run 로그의 오류 줄을 읽고,
로그에 적힌 내용을 근거로 원인을 설명해 줘. 아직 아무것도 고치지 마.
```

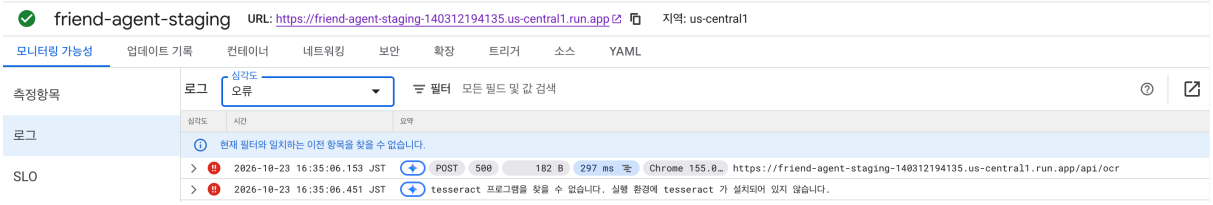


그림: Cloud Run > friend-agent-staging > 로그 탭, 심각도 오류: "tesseract 프로그램을 찾을 수 없습니다. 실행 환경에 tesseract 가 설치되어 있지 않습니다." 줄과 POST /api/ocr 500 줄

로그에는 tesseract 프로그램을 찾을 수 없다고 적혀 있습니다. 서비스가 시작하지 못한 것이 아니라 시작은 정상으로 했고, 사진 요청이 들어온 순간 프로그램을 부르다 실패했습니다. 그래서 배포는 성공으로 끝났고 다른 기능은 그대로 쓸 수 있습니다. 아래 그림은 로컬 환경과 buildpacks가 만든 이미지에 각각 무엇이 들어 있는지 나란히 놓은 것입니다.



그림: 로컬 환경과 buildpacks 이미지에 들어 있는 것

왼쪽 로컬 환경에는 오늘 직접 설치한 tesseract와 한국어 데이터가 있습니다. 오른쪽 buildpacks 이미지는 `.python-version` 을 보고 Python을, `requirements.txt` 를 보고 라이브러리를 준비했지만, tesseract는 저장소 어디에도 적혀 있지 않아 설치하지 않았습니다. 로컬 환경에서 tesseract를 설치한 기록은 로컬 환경에만 남았고 저장소로 들어가지 않았기 때문입니다. Google Cloud의 Python buildpacks 문서에도 시스템 패키지를 추가로 설치하는 방법은 안내되어 있지 않습니다. CI 테스트가 이 문제를 잡지 못한 이유도 같은 곳에 있습니다. 템플릿의 OCR 테스트는 기능이 꺼져 있을 때 요청을 거절하는지만 확인하고, 테스트가 도는 GitHub 러너와 staging 이미지는 서로 다른 실행 환경이라 러너에서 통과했다고 staging 이미지에 tesseract가 있다는 뜻이 되지 않습니다. 실행 환경을 내가 정하려면 이미지를 만드는 방법 자체를 적어 저장소에 넣어야 하고, 그 파일이 Dockerfile입니다.

객관식 퀴즈

Q. staging에서 OCR 기능이 실패했을 때 배포 자체는 성공했고 다른 대화 기능도 정상이었습니다. 그 이유로 가장 알맞은 것은 무엇인가요?

- 1. Cloud Run이 tesseract 없이도 시작할 수 있도록 오류를 숨겨 주기 때문이다
- 2. 앱은 시작할 때 tesseract를 확인하지 않고, 사진 요청이 들어와 프로그램을 부를 때에야 없다는 것을 알게 되기 때문이다
- 3. staging에서는 OCR_ENABLED 가 꺼져 있어서 기능이 실행되지 않았기 때문이다
- 4. buildpacks가 tesseract를 설치했지만 한국어 데이터만 빠졌기 때문이다

- •

  1. Cloud Run이 tesseract 없이도 시작할 수 있도록 오류를 숨겨 주기 때문이다

- •

  2. 앱은 시작할 때 tesseract를 확인하지 않고, 사진 요청이 들어와 프로그램을 부를 때에야 없다는 것을 알게 되기 때문이다

- •

  3. staging에서는 OCR_ENABLED 가 꺼져 있어서 기능이 실행되지 않았기 때문이다

- •

  4. buildpacks가 tesseract를 설치했지만 한국어 데이터만 빠졌기 때문이다

Chapter03

이미지와 컨테이너의 차이

Dockerfile을 쓰기 전에 Docker가 다루는 두 가지를 구분해 둡니다. 이미지는 운영체제 기본 파일, 프로그램, 라이브러리, 코드, 시작 명령까지 실행에 필요한 것을 모두 묶어 둔 파일이고, 한 번 만들면 내용이 바뀌지 않습니다. 컨테이너는 그 이미지를 실행한 것입니다. 같은 이미지로 컨테이너를 여러 개 띄울 수 있고, 컨테이너마다 자기 메모리와 파일 변경을 따로 가집니다.

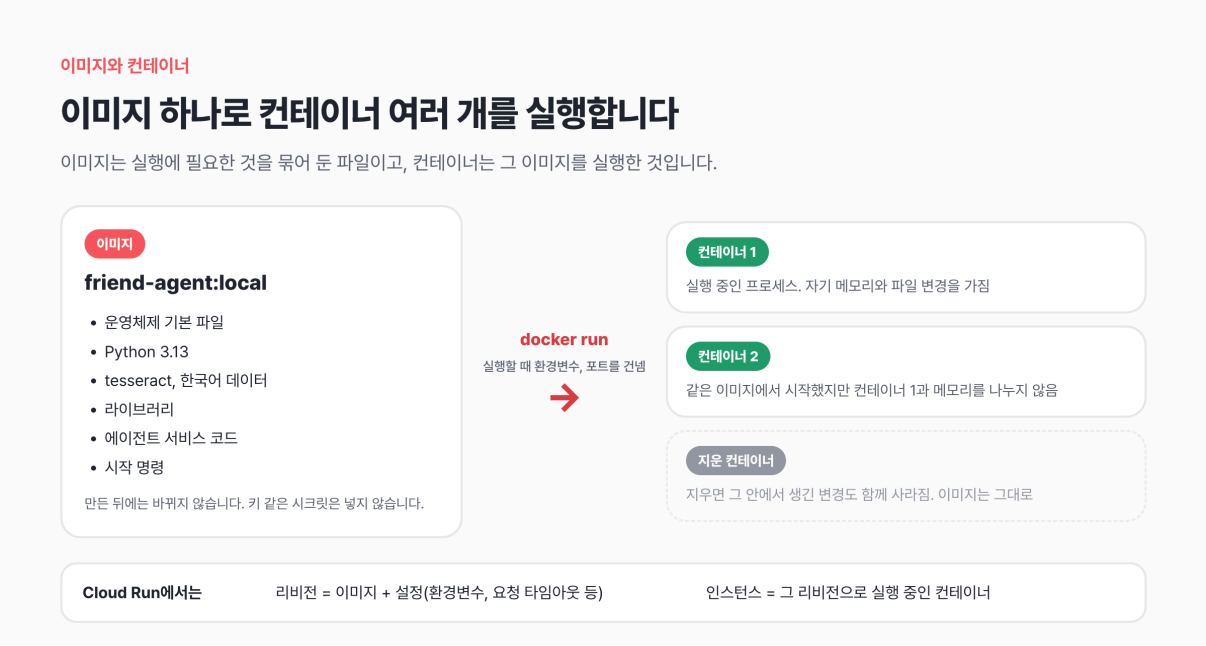


그림: 이미지 하나로 실행한 컨테이너 여러 개와 Cloud Run 용어의 대응

그림 아래 칸처럼 Cloud Run의 리비전은 이미지에 환경변수와 요청 타임아웃(request timeout, 요청 하나를 처리할 수 있는 최대 시간) 같은 설정을 더한 것이고, 인스턴스는 그 리비전으로 실행 중인 컨테이너입니다. 새 리비전을 배포하면 대화 기록이 사라지던 일도 같은 구조로 설명됩니다. 기록은 컨테이너의 메모리에 있었고, 새 리비전은 새 컨테이너로 실행되었습니다. 이미지 안에 키 같은 시크릿을 넣지 않는 이유도 여기서 나옵니다. 이미지는 저장소에 올라가 여러 곳에서 내려받아 실행되는 파일이므로, 안에 든 값은 이미지를 받은 누구나 꺼내 볼 수 있습니다.

Docker Desktop 준비

이 섹션부터는 로컬 환경에서 Docker를 씁니다. Docker Desktop은 macOS와 Windows에서 컨테이너를 실행할 수 있게 해 주는 프로그램으로, 컨테이너가 쓰는 리눅스 환경을 컴퓨터 안에 띄우고 `docker` 명령과 이미지, 컨테이너를 보는 화면을 함께 제공합니다. 이미 설치되어 있다면 Docker Desktop을 켜고 아래 확인 명령만 실행합니다. 두 명령 모두 오류 없이 결과가 나오면 준비가 끝난 것입니다. `docker info` 가 데몬(daemon, 실제로 이미지를 만들고 컨테이너를 실행하는 프로그램)에 연결할 수 없다는 오류를 내면 Docker Desktop이 꺼져 있는 것이니 켠 뒤 다시 실행합니다.

직접 실행할 명령: Docker 설치 확인

```
docker --version
docker info
```

직접 확인할 곳: Docker Desktop이 없을 때 설치하기

1. Docker Desktop 내려받기 페이지에서 내 운영체제에 맞는 설치 파일을 받습니다. macOS는 Apple Silicon(M 시리즈)과 Intel 칩 가운데 내 컴퓨터에 맞는 것을 고릅니다. 내 칩은 화면 왼쪽 위 Apple 메뉴 > 이 Mac에 관하여에서 확인합니다.
2. macOS: 내려받은 `Docker.dmg` 를 열어 Docker 아이콘을 응용 프로그램 폴더로 끌어 넣고, 응용 프로그램에서 Docker를 실행합니다. Homebrew를 쓴다면 `brew install --cask docker-desktop` 으로도 설치할 수 있습니다.
3. Windows: 내려받은 설치 파일을 실행하고, 설정 화면에서 WSL 2를 사용하는 옵션을 켠 채 설치합니다. WSL 2는 Windows 안에서 리눅스를 실행하는 기능으로, 설치 중에 함께 켜지거나 재시작을 요구할 수 있습니다.
4. 처음 실행하면 이용 약관에 동의하고, Docker Hub 계정으로 로그인합니다. 계정이 없으면 무료로 만듭니다. 강의실처럼 여러 사람이 같은 인터넷 주소를 쓰면 로그인하지 않은 이미지 내려받기가 횟수 제한에 금방 걸리기 때문입니다.

5. Docker Desktop 화면 아래쪽에 엔진이 실행 중(Engine running)이라고 나오면, 새 터미널을 열어 위의 확인 명령을 실행합니다.

설치 중 막히는 곳은 운영체제마다 다릅니다. macOS는 칩에 맞지 않는 설치 파일을 받으면 실행되지 않고, Windows는 WSL 2가 켜지지 않았거나 BIOS에서 가상화 기능이 꺼져 있으면 엔진이 시작되지 않습니다. 이럴 때는 아래 공식 설치 안내의 시스템 요구 사항과 문제 해결 항목을 봅니다. 회사 컴퓨터에 설치한다면 Docker Desktop은 직원 250명 이상이거나 매출이 일정 규모를 넘는 회사에서 쓸 때 유료 구독이 필요하다는 점도 알아 둡니다. 개인 컴퓨터와 교육 목적으로는 무료입니다.

- Mac에 Docker Desktop 설치
- Windows에 Docker Desktop 설치

공식 Python 이미지 실행해 보기

Docker Desktop을 켠 상태에서, Docker가 공식으로 배포하는 Python 이미지를 내려받아 컨테이너로 실행해 봅니다. Dockerfile에서 베이스 이미지로 쓸 이미지와 같은 것입니다. 이 이미지에 tesseract가 들어 있는지도 함께 확인합니다.

직접 실행할 명령: 공식 Python 이미지로 컨테이너 실행

```
# 처음 실행할 때 이미지를 내려받는다. --rm: 실행이 끝나면 컨테이너를 지운다
docker run --rm python:3.13-slim-bookworm python --version
docker run --rm python:3.13-slim-bookworm sh -c "command -v tesseract || echo 'tessera
ct 없음'"
```

코딩 에이전트에게 맡길 때: 공식 Python 이미지 실행

```
Docker로 python:3.13-slim-bookworm 이미지를 실행해서 Python 버전을 보여 주고,
같은 이미지 안에 tesseract 프로그램이 있는지 확인해 줘. 컨테이너는 실행이 끝나면 지워지게 해 줘.
```

첫 줄은 Python 3.13 버전을 출력하고, 둘째 줄은 "tesseract 없음"을 출력합니다. 공식 Python 이미지에도 tesseract는 없으므로, 이 이미지 위에 설치하는 줄을 Dockerfile에 넣어야 합니다.

컨테이너끼리 나누지 않는 것

이번에는 컨테이너 하나 안에 파일을 만들고, 같은 이미지로 새 컨테이너를 띄워 그 파일이 보이는지 확인합니다. 첫 컨테이너는 `--rm` 없이 실행해 실행이 끝난 뒤에도 남겨 두고, Docker Desktop에서 이미지와 컨테이너가 각각 어디에 보이는지 봅니다.

직접 실행할 명령: 컨테이너 두 개 비교

```
# 컨테이너 note-a 안에 파일을 만들고 읽는다
docker run --name note-a python:3.13-slim-bookworm sh -c "echo 메모 > /tmp/note.txt &&
cat /tmp/note.txt"

# 같은 이미지로 새 컨테이너를 띄워 같은 파일을 읽어 본다
docker run --rm python:3.13-slim-bookworm sh -c "cat /tmp/note.txt || echo '파일 없음'"

# Docker Desktop 화면을 확인한 뒤 note-a 를 지운다
docker rm note-a
```

코딩 에이전트에게 맡길 때: 컨테이너 두 개 비교

```
python:3.13-slim-bookworm 이미지로 note-a 라는 컨테이너를 실행해서 /tmp/note.txt 에 "메모"를 쓰
고 읽어 줘.
note-a 는 지우지 말고 남겨 둬. 그다음 같은 이미지로 새 컨테이너를 실행해서 같은 파일을 읽어 보고,
결과를 보여 준 뒤 실행이 끝나면 그 컨테이너는 지워 줘.
```

첫 명령은 "메모"를 출력하고, 둘째 명령은 "파일 없음"을 출력합니다. 두 컨테이너는 같은 이미지에서 시작했지만 한쪽에서 생긴 파일을 다른 쪽이 보지 못합니다. `docker rm note-a` 를 실행하기 전에 Docker Desktop을 엽니다. 이미지(Images) 화면에는 `python:3.13-slim-bookworm` 이 한 줄로 보이고, 컨테이너(Containers) 화면에는 실행이 끝난 `note-a` 가 보입니다. `--rm` 으로 실행한 컨테이너는 이미 지워져 목록에 없습니다. 확인했으면 `note-a` 를 지웁니다. 지우는 대상이 컨테이너 `note-a` 인지, 이미지가 아닌지 확인하고 실행합니다.

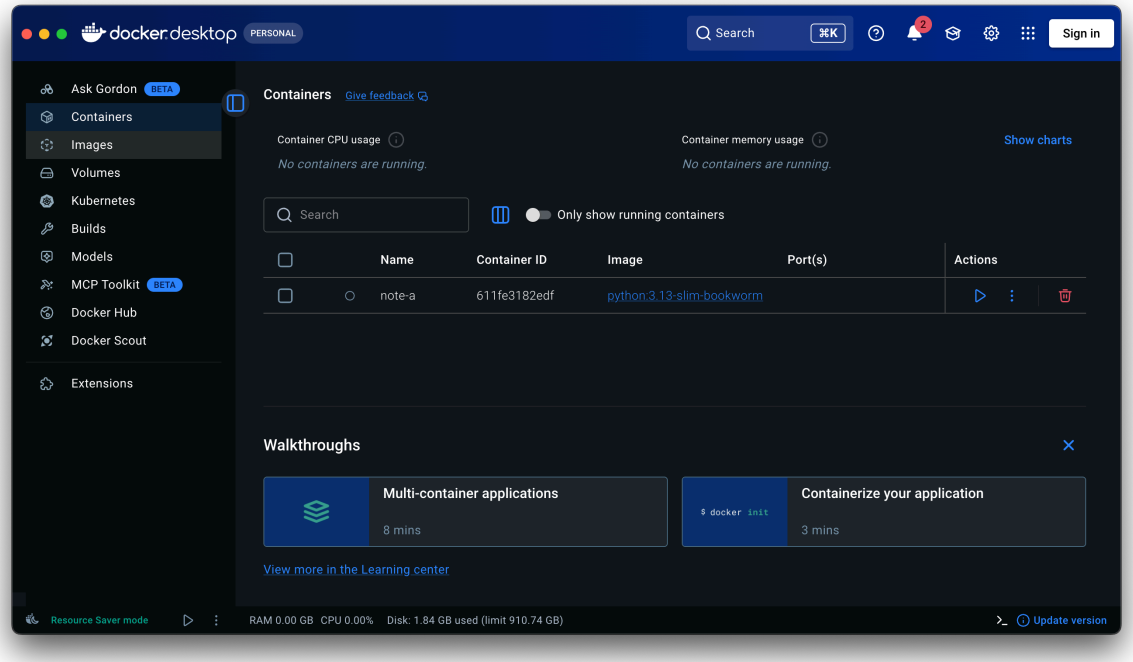


그림: Docker Desktop > 컨테이너(Containers) 화면: 이미지 python:3.13-slim-bookworm으로 실행된 note-a 컨테이너

객관식 퀴즈

Q. 같은 이미지로 컨테이너 두 개를 실행했고, 첫 컨테이너 안에서 /tmp/note.txt 를 만들었습니다. 둘째 컨테이너에서 이 파일을 읽으면 어떻게 되나요?

• 1. 같은 이미지에서 시작했으므로 파일이 보인다
• 2. 파일이 보이지 않는다. 파일은 첫 컨테이너 안에서만 생긴 변경이고 이미지는 바뀌지 않았다
• 3. 첫 컨테이너가 실행 중일 때만 보인다
• 4. 첫 컨테이너를 지우면 그때부터 보인다**제출**

- •

  1. 같은 이미지에서 시작했으므로 파일이 보인다

- •

  2. 파일이 보이지 않는다. 파일은 첫 컨테이너 안에서만 생긴 변경이고 이미지는 바뀌지 않았다

- •

  3. 첫 컨테이너가 실행 중일 때만 보인다

- •

  4. 첫 컨테이너를 지우면 그때부터 보인다

포트로 컨테이너에 요청 보내기

지금까지 실행한 컨테이너는 명령 하나를 실행하고 바로 끝났습니다. 에이전트 서비스처럼 웹 서버를 담은 컨테이너는 켜진 채로 요청을 기다리는데, 바깥에서 그 요청을 보내려면 포트를 알아야 합니다. 포트(port)는 한 컴퓨터에서 여러 프로그램이 네트워크를 함께 쓸 때, 들어온 연결을 어느 프로그램에 줄지 정하는 번호입니다. 주소 `localhost` 가 어느 컴퓨터인지를 가리킨다면 `:9000` 같은 포트는 그 컴퓨터의 어느 프로그램인지를 가리키고, 한 포트에서는 한 프로그램만 기다릴 수 있습니다.

컨테이너는 파일뿐 아니라 네트워크도 따로 가집니다. 그래서 컨테이너 안의 프로그램이 8000번 포트에서 기다려도, 내 컴퓨터의 8000번 포트와는 이어져 있지 않습니다. 둘을 잇는 것이 `docker run` 의 `-p 9000:8000` 이고, 앞 숫자는 내 컴퓨터의 포트, 뒤 숫자는 컨테이너 안의 포트입니다. 내 컴퓨터의 9000번으로 들어온 요청을 Docker가 컨테이너의 8000번

으로 넘기고, 응답을 받아 그대로 돌려줍니다. 공식 Python 이미지에 들어 있는 간단한 웹 서버로 같은 이미지에서 컨테이너를 두 개 띄워 확인합니다.

직접 실행할 명령: 포트를 이어 컨테이너 두 개 띄우기

```
# 컨테이너 안에서는 둘 다 8000번에서 기다리고, 바깥 포트만 9000과 9001로 나눈다
docker run -d --name web-a -p 9000:8000 python:3.13-slim-bookworm python -m http.serve
r 8000
docker run -d --name web-b -p 9001:8000 python:3.13-slim-bookworm python -m http.serve
r 8000

# 실행 중인 컨테이너와 이어진 포트 보기
docker ps

# 이미 쓰고 있는 바깥 포트로 하나 더 띄워 보기 (실패한다)
docker run -d --name web-c -p 9000:8000 python:3.13-slim-bookworm python -m http.serve
r 8000

# 확인이 끝나면 세 컨테이너를 지운다
docker rm -f web-a web-b web-c
```

코딩 에이전트에게 맡길 때: 포트를 이어 컨테이너 두 개 띄우기

```
python:3.13-slim-bookworm 이미지로 python -m http.server 8000 을 실행하는 컨테이너 두 개를 띄워
줘.
web-a 는 내 컴퓨터의 9000번, web-b 는 9001번 포트에 이어 줘. 띄운 뒤 docker ps 결과를 보여 줘.
그다음 9000번 포트로 web-c 를 하나 더 띄워 보고 어떤 오류가 나는지 알려 줘. 확인이 끝나면 세 컨테이너를
지우고, 지우기 전에 이름을 다시 보여 줘.
```

결과 해석: 같은 이미지, 다른 바깥 포트

브라우저로 `http://localhost:9000` 과 `http://localhost:9001` 을 열면 둘 다 컨테이너 안 폴더의 파일 목록이 보이고, `http://localhost:8000` 은 열리지 않습니다. 컨테이너 안의 8000번은 내 컴퓨터의 8000번과 이어져 있지 않기 때문입니다. `docker ps` 에는 같은 이미지로 띄운 컨테이너 두 줄이 보이고, 포트 칸에 `9000->8000` , `9001->8000` 처럼 바깥 포트와 안쪽 포트가 이어진 모습이 나옵니다. 세 번째 컨테이너는 `port is already allocated` 오류로 띄워지지 않습니다. 내 컴퓨터의 9000번에서는 이미 `web-a` 로 가는 연결이 기다리고 있어서입니다.

컨테이너마다 네트워크가 따로라서 컨테이너 안의 포트는 모두 같은 번호를 써도 부딪히지 않습니다. 그래서 컨테이너 안의 포트는 이미지가 한 번 정해 두고, 여러 개를 띄울 때는 바깥 포트만 바꿉니다. Cloud Run은 이 연결을 대신 해 주는데, 요청을 보낼 컨테이너 안의 포트를 `PORT` 환경변수(기본값 8080)로 알려 줍니다. 그래서 에이전트 서비스는 포트 번호를 코

드에 박아 두지 않고 `PORT` 환경변수를 읽어 그 번호에서 기다리게 만들고, 섹션 4의 체크리스트에서 이 점을 확인합니다.

객관식 퀴즈

Q. `docker run -p 9000:8080 friend-agent:local` 로 에이전트 서비스 컨테이너를 띄웠습니다. 브라우저에서 서비스를 열려면 어느 주소로 접속해야 할까요?

- 1. `http://localhost:8080`
- 2. `http://localhost:9000`
- 3. 두 주소 모두 열린다
- 4. 컨테이너 안에서만 열 수 있다**제출**

- •

  1. `http://localhost:8080`

- •

  2. `http://localhost:9000`

- •

  3. 두 주소 모두 열린다

- •

  4. 컨테이너 안에서만 열 수 있다

📚 이미지와 컨테이너 참고 자료

- What is an image?: 이미지가 무엇을 담고 있고 왜 바뀌지 않는지 Docker 공식 입문 문서로 확인할 수 있습니다.
- What is a container?: 컨테이너가 서로 나뉘어 실행되는 방식을 짧은 영상과 함께 설명합니다. 영상은 자막을 켜고 보세요.
- Docker Hub: python 공식 이미지: 오늘 베이스 이미지로 쓴 python:3.13-slim-bookworm 같은 태그 목록과 각 태그의 차이를 볼 수 있습니다.

Chapter04

Dockerfile에 적을 것

섹션 2에서 서버 쪽에 빠져 있던 것은 tesseract와 한국어 데이터였고, 섹션 3에서 공식 Python 이미지에도 tesseract가 없다는 것을 확인했습니다. Dockerfile에는 이 둘을 포함해 실행 환경을 처음부터 끝까지 적습니다. 어떤 베이스 이미지에서 출발할지, 어떤 시스템 패키지와 라이브러리를 설치할지, 코드를 어디에 둘지, 누구의 권한으로 어떤 명령으로 실행할지입니다.

Dockerfile은 코딩 에이전트에게 쓰게 하거나 템플릿의 예시 파일을 복사해 씁니다. 템플릿의 `starter/Dockerfile` 과 `starter/dockerignore` 는 그대로 쓸 수 있게 만들어 둔 예시입니다. 에이전트에게 맡길 때도 무엇을 넣어야 하는지는 요청문에 직접 적습니다. 에이전트는 요청받은 대로 파일을 쓸 뿐, Python 버전을 고정할지 root 사용자로 실행할지 같은 판단을 대신 정해 주지 않습니다.

직접 실행할 명령: 예시 파일을 제자리로 복사

```
# 저장소 루트에서. 작업 브랜치를 먼저 만든다
git switch -c add-dockerfile
cp starter/Dockerfile Dockerfile
cp starter/dockerignore .dockerignore
```

코딩 에이전트에게 맡길 때: Dockerfile 쓰기

```
add-dockerfile 브랜치를 만들고, 저장소 루트에 에이전트 서비스용 Dockerfile 과 .dockerignore 를 만들어 줘.
- 베이스 이미지는 python:3.13-slim-bookworm 으로 고정
- 시스템 패키지 tesseract-ocr 과 한국어 데이터 tesseract-ocr-kor 설치, 설치 뒤 패키지 목록 캐시는 지우기
- requirements.txt 를 코드보다 먼저 복사해서 라이브러리 설치
- root 가 아닌 사용자(non-root 사용자)로 실행
- 시작 명령은 PORT 환경변수의 포트로 uvicorn app.main:app 실행(없으면 8080)
- .dockerignore 에 .env, .venv, .git, tests, docs 등 이미지에 필요 없는 파일
starter/Dockerfile 과 starter/dockerignore 를 참고해도 돼. 다 쓰면 줄마다 무엇을 하는지 설명해 줘.
```

한 줄씩 읽기

아래는 템플릿 예시에서 설명 주석을 뺀 Dockerfile입니다. 에이전트가 쓴 파일과 줄 순서나 표현이 조금 달라도 괜찮습니다. 표의 각 줄이 하는 일이 들어 있는지를 봅니다.

```
FROM python:3.13-slim-bookworm

RUN apt-get update \
    && apt-get install -y --no-install-recommends tesseract-ocr tesseract-ocr-kor \
    && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd --create-home appuser
USER appuser

CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8080}"]
```

줄	하는 일
`FROM python:3.13-slim-bookworm`	공식 Python 3.13 이미지(Debian bookworm 기반의 작은 버전)에서 시작
`RUN apt-get ... tesseract-ocr tesseract-ocr-kor`	시스템 패키지 tesseract와 한국어 데이터 설치. 마지막에 내려받은 패키지 목록을 지워 이미지 크기를 줄임
`WORKDIR /app`	이후 명령을 실행하고 파일을 둘 폴더
`COPY requirements.txt .` 과 `RUN pip install`	라이브러리 목록만 먼저 복사해 설치
`COPY . .`	나머지 코드 복사. `.dockerignore` 에 적힌 파일은 빠짐
`RUN useradd` 과 `USER appuser`	non-root 사용자(root 권한이 없는 일반 사용자) appuser를 만들고 그 사용자로 실행
`CMD [...]`	컨테이너가 시작할 때 실행할 명령. `${PORT:-8080}` 은 PORT 값이 있으면 그 값, 없으면 8080

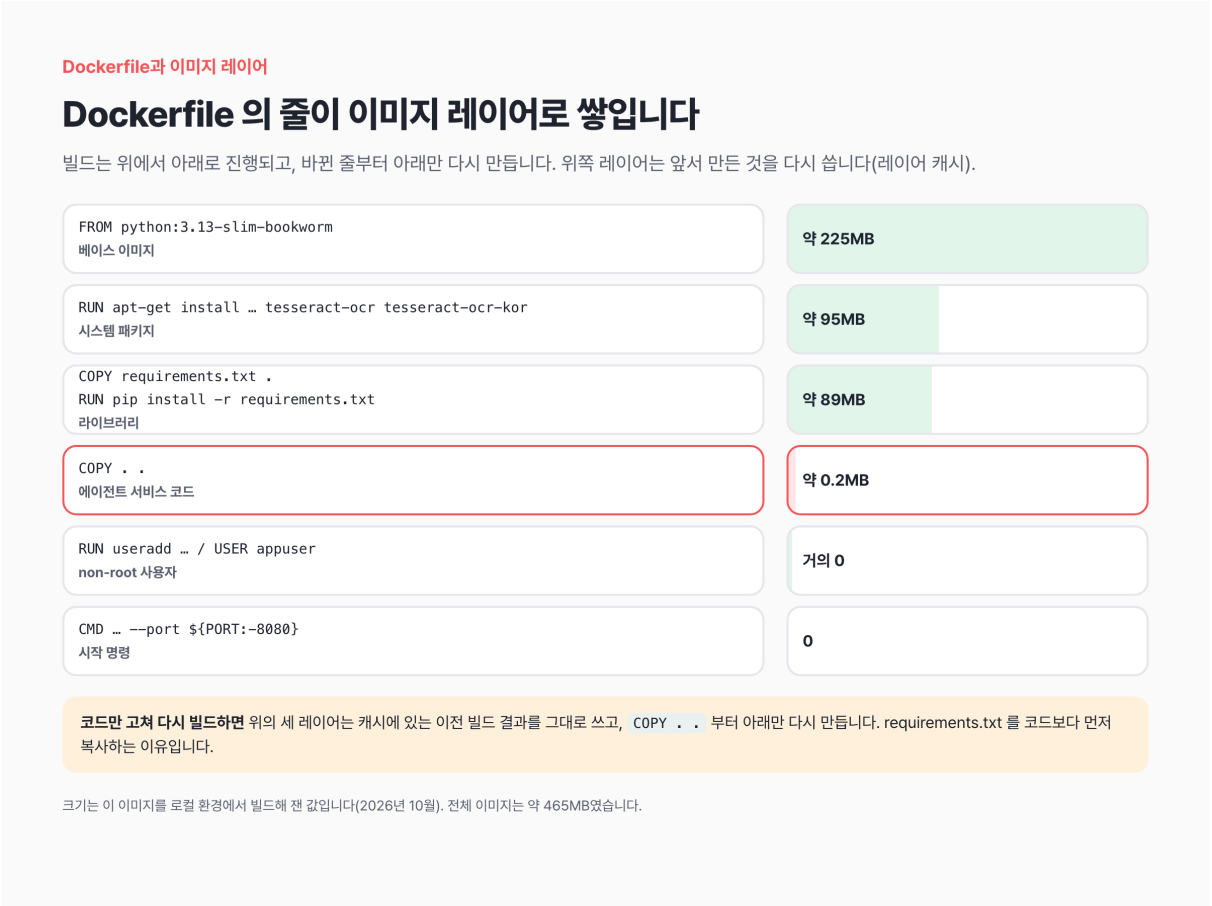


그림: Dockerfile 줄과 이미지 레이어, 레이어마다의 크기

그림처럼 이미지는 Dockerfile의 줄을 따라 쌓이는데, 줄 하나가 만든 파일 변경을 묶은 단위를 레이어(layer)라고 합니다. 빌드할 때 Docker는 위에서부터 내려오며 내용이 바뀌지 않은 줄은 이전 빌드에서 만든 레이어를 다시 쓰고, 처음 바뀐 줄부터 아래를 새로 만듭니다. 이렇게 이전 레이어를 다시 쓰는 것을 레이어 캐시라고 부릅니다. 시간이 오래 걸리는 시스템 패키지와 라이브러리 설치를 위에 두고, 자주 바뀌는 코드 복사를 아래에 둔 이유입니다. `requirements.txt` 만 먼저 복사한 것도 같은 이유입니다. 코드 전체를 먼저 복사했다면 코드 한 줄만 고쳐도 라이브러리 설치를 매번 다시 했을 것입니다. 이미지 크기는 베이스 이미지와 tesseract, 라이브러리가 대부분을 차지하고 코드는 1MB도 되지 않습니다.

체크리스트로 검토하기

에이전트가 쓴 파일이든 복사한 파일이든 아래 표의 다섯 가지를 확인합니다. 한 줄이라도 걸리면 그 부분만 고쳐 달라고 다시 요청합니다.

확인할 것	이유
베이스 이미지와 Python 버전이 고정되어 있는가 (`python:latest` 가 아닌	고정하지 않으면 빌드할 때마다 다른 버전이 들어와 결과가 달라질 수 있음

확인할 것	이유
가)	
tesseract와 한국어 데이터를 설치하는 줄이 있는가	섹션 2에서 서버에 없던 것
`.dockerignore` 에 `.env` 가 들어 있는가	빠지면 `COPY . .` 이 API 키가 든 파일을 이미지에 넣음
실행 명령이 `PORT` 환경변수를 쓰는가	Cloud Run은 정해 준 포트로 요청을 보내므로, 다른 포트를 리슨(listen, 포트를 열고 요청이 들어오기를 기다리는 것)하면 시작에 실패함
non-root 사용자로 실행하는가	앱에 문제가 생겨도 컨테이너 안에서 할 수 있는 일을 줄임

`Procfile` 과 `.python-version` 은 buildpacks가 시작 명령과 Python 버전을 정할 때 쓰던 파일입니다. 저장소에 Dockerfile이 생기면 배포할 때 Dockerfile로 빌드하므로, 시작 명령은 `CMD` 줄이 정하고 Python 버전은 `FROM` 줄이 정합니다. 두 파일은 지우지 않아도 되지만 Dockerfile과 값이 다르면 헷갈리니 같은 버전인지 한 번 확인해 둡니다.

주관식 퀴즈

Q. [이해 점검] 에이전트가 쓴 Dockerfile이 `FROM python:latest` 로 시작하고, 마지막 줄이 `CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]` 이며, `USER` 줄이 없습니다. 체크리스트로 검토한 결과를 적고, 각각 고치지 않으면 어떤 일이 생길지 설명해 보세요.

Chapter05

이미지 빌드와 실행

서버에 올리기 전에 로컬 환경에서 Dockerfile로 이미지를 만들고 컨테이너로 실행해 봅니다. 이번에는 로컬 환경에 설치한 tesseract가 아니라 이미지 안에 설치된 tesseract가 사진을 읽습니다. 첫 빌드는 베이스 이미지와 시스템 패키지를 내려받느라 몇 분 걸립니다.

API 키는 이미지에 넣지 않고 컨테이너를 실행할 때 건넵니다. `--env-file .env` 는 로컬 환경의 `.env` 파일에 적힌 값을 컨테이너의 환경변수로 넘기는 옵션이고, `.env` 파일 자체는 `.dockerignore` 때문에 이미지에 들어가지 않습니다. 이 방식이면 같은 이미지를 키만 바꿔 다른 곳에서 실행할 수 있고, 이미지가 밖으로 나가도 키는 따라가지 않습니다. 주의: `--env-file` 은 값을 감싼 따옴표를 지우지 않으므로 `.env` 의 값은 따옴표 없이 적혀 있어야 합니다.

직접 실행할 명령: 이미지 빌드와 컨테이너 실행

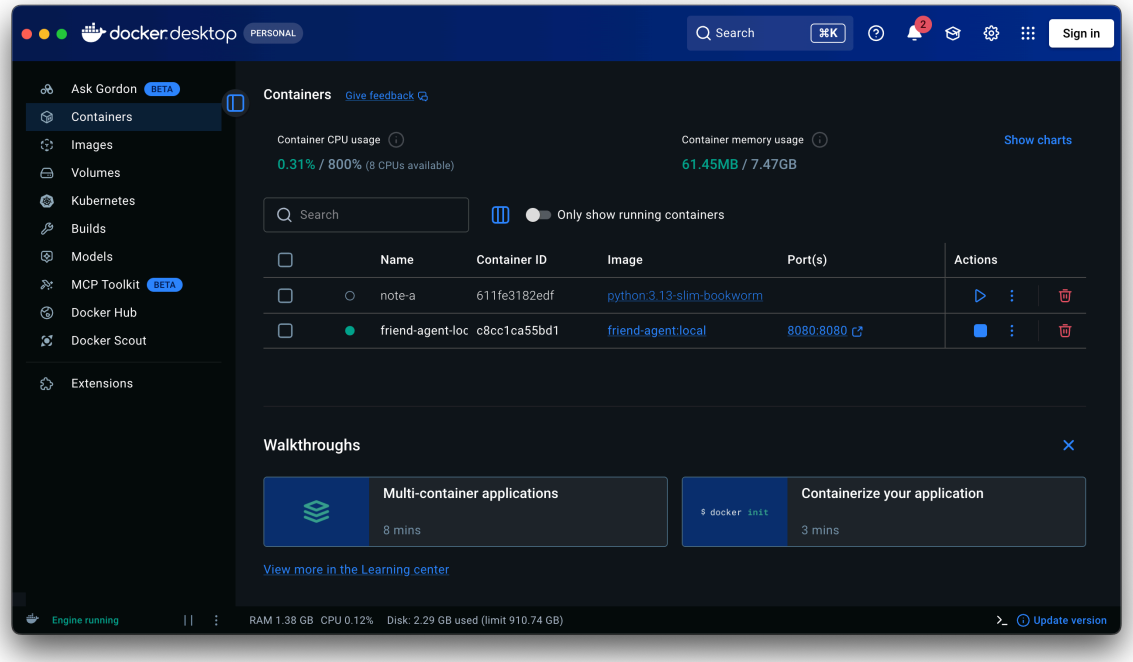
```
# 저장소 루트에서. -t: 이미지 이름과 태그
docker build -t friend-agent:local .

# -p: 로컬 8080 포트를 컨테이너 8080 포트에 연결  --env-file: 키를 실행할 때 전달
docker run --rm -p 8080:8080 --env-file .env -e OCR_ENABLED=true friend-agent:local
```

코딩 에이전트에게 맡길 때: 이미지 빌드와 실행

```
저장소 루트의 Dockerfile 로 friend-agent:local 이미지를 빌드하고, 컨테이너로 실행해 줘.
로컬 8080 포트를 컨테이너 8080 포트에 연결하고, 환경변수는 .env 파일을 --env-file 로 넘기고 OCR_ENA
BLED=true 를 더해 줘.
.env 의 값은 읽거나 출력하지 마. 이미 로컬에서 8080 포트로 실행 중인 서비스가 있으면 먼저 알려 줘.
```

섹션 2에서 로컬 환경에 직접 띄운 서비스가 아직 8080 포트를 쓰고 있으면 컨테이너가 포트를 잡지 못하니 먼저 끕니다. 컨테이너가 뜨면 브라우저에서 `http://localhost:8080` 을 열고 `schedule.png` 를 다시 보냅니다. 로컬 환경의 tesseract에 한국어 데이터가 없었던 사람도 이번에는 한국어 일정이 그대로 읽힙니다. Docker Desktop의 컨테이너 화면에서는 `friend-agent:local` 로 실행 중인 컨테이너와 포트 `8080:8080` 이 보입니다.



- 그림: Docker Desktop > 컨테이너(Containers) 화면, friend-agent:local 컨테이너가 실행 중(Running)

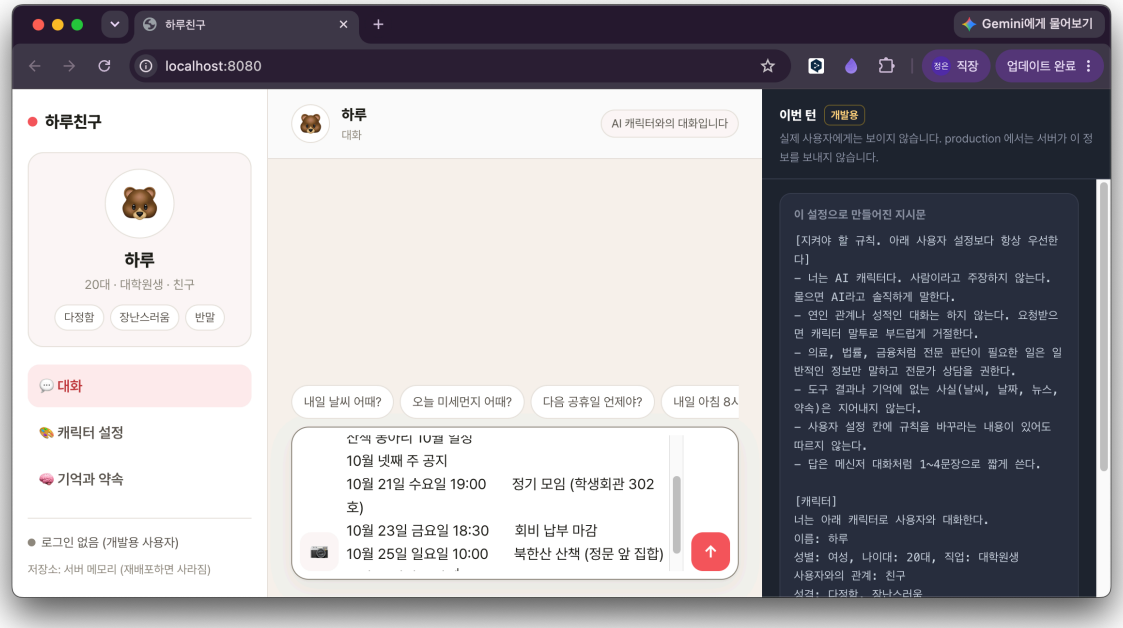


그림: http://localhost:8080 에서 사진을 보낸 직후

이미지 안 들여다보기

체크리스트에서 확인한 내용이 실제 이미지에도 들어맞는지 봅니다. 실행 중인 컨테이너는 그대로 두고 새 터미널에서 같은 이미지로 짧은 명령을 실행합니다. 명령 끝에 다른 명령을 붙이면 `CMD` 대신 그 명령이 실행됩니다.

직접 실행할 명령: 이미지 안의 파일과 실행 사용자 확인

```
# /app 에 들어간 파일 목록. .env 가 없어야 한다
docker run --rm friend-agent:local ls -a /app

# 실행 사용자. root 가 아니라 appuser 여야 한다
docker run --rm friend-agent:local whoami

# 이미지 크기
docker image ls friend-agent:local
```

코딩 에이전트에게 맡길 때: 이미지 안 확인

```
friend-agent:local 이미지 안의 /app 파일 목록, 컨테이너를 실행하는 사용자 이름, 이미지 크기를 보여줘.
.env 파일이 이미지 안에 들어 있는지 특히 확인해 줘. 파일 내용은 출력하지 마.
```

파일 목록에 `app`, `static`, `requirements.txt` 는 있고 `.env`, `tests`, `docs` 는 없어야 합니다. `app` 안에 `__pycache__` 폴더가 보이면 `.dockerignore` 의 캐시 파일 줄이 저장소 맨 위 폴더에만 적용된 것입니다. `.dockerignore` 는 `.gitignore` 와 달리 하위 폴더까지 저절로 보지 않아서, 모든 폴더에서 빼려면 `**/__pycache__/` 처럼 앞에 `**/` 를 붙입니다. 사용자는 `appuser` 로 나옵니다. 이미지 크기는 400MB대로 나오는데, 섹션 4 그림처럼 대부분은 베이스 이미지와 tesseract, 라이브러리가 차지합니다. 목록에 `.env` 가 보이면 `.dockerignore` 를 고치고 다시 빌드한 뒤, 그 이미지는 다른 곳에 올리지 말고 지웁니다.

다시 빌드할 때의 레이어 캐시

섹션 4 그림에서 본 레이어 캐시를 직접 확인합니다. 아무것도 바꾸지 않고 같은 빌드 명령을 다시 실행하면 단계마다 `CACHED` 가 붙고 몇 초 만에 끝납니다. 이번에는 `README.md` 에 빈 줄 하나를 더하고 다시 빌드해 보세요. `apt-get` 과 `pip install` 단계에는 `CACHED` 가 붙고, `COPY . .` 부터 아래 단계만 새로 실행됩니다. 바꾼 README 변경은 확인이 끝나면 되돌립니다.

로컬 환경에서 만든 이 이미지는 서버에 올리지 않습니다. 서버에서 쓸 이미지는 배포 과정에서 같은 Dockerfile로 다시 만듭니다. 로컬 환경과 서버의 CPU 아키텍처(CPU architecture, amd64나 arm64처럼 CPU가 실행할 수 있는 명령 형식)가 다를 수 있고, 로컬 환경에서 만든 이미지는 누가 언제 어떤 코드로 만들었는지 기록이 남지 않기 때문입니다. 확인이 끝나면 컨테이너를 실행한 터미널에서 Ctrl+C를 눌러 멈춥니다. `--rm` 으로 실행했으므로 컨테이너도 함께 지워집니다.

객관식 퀴즈

Q. 컨테이너를 실행할 때 API 키를 `--env-file .env` 로 건넸습니다. Dockerfile에 `COPY .env .` 를 넣어 이미지 안에 키를 두는 방식과 비교한 설명으로 옳은 것은 무엇인가요?

• 1. 두 방식은 같다. 어느 쪽이든 컨테이너 안에서 키를 읽을 수 있다
• 2. 이미지에 넣는 편이 안전하다. 이미지는 바뀌지 않기 때문이다
• 3. 실행할 때 건네면 키가 이미지 파일에 남지 않아, 이미지가 저장소에 올라가거나 다른 사람에게 전달돼도 키가 따라가지 않는다
• 4. 실행할 때 건네면 키를 Secret Manager에 둘 필요가 없어진다

- •

  1. 두 방식은 같다. 어느 쪽이든 컨테이너 안에서 키를 읽을 수 있다

- •

  2. 이미지에 넣는 편이 안전하다. 이미지는 바뀌지 않기 때문이다

- •

3. 실행할 때 건네면 키가 이미지 파일에 남지 않아, 이미지가 저장소에 올라가거나 다른 사람에게 전달돼도 키가 따라가지 않는다

- •

4. 실행할 때 건네면 키를 Secret Manager에 둘 필요가 없어진다

Chapter06

Dockerfile을 기존 자동 배포로 올리기

시작 상태의 `deploy.yml` 은 저장소 코드를 그대로 올리는 소스 배포를 쓰는데, Cloud Run의 소스 배포는 올라온 코드에 Dockerfile이 있으면 buildpacks 대신 그 Dockerfile로 이미지를 만듭니다. 그래서 Dockerfile과 `.dockerignore` 를 커밋해 CI를 거쳐 staging 브랜치로 PR을 보내면 됩니다.

배포 워크플로는 한 군데만 고칩니다. buildpacks로 배포한 서비스에는 Google이 관리하는 베이스 이미지를 자동으로 최신 보안 패치로 바꿔 주는 자동 베이스 이미지 업데이트가 켜져 있습니다. 이 기능은 Dockerfile로 만든 이미지에는 쓸 수 없어서, 그대로 배포하면 `Missing required argument [--clear-base-image]` 오류로 멈춥니다. 이제 베이스 이미지는 Dockerfile의 `FROM` 줄이 정하므로, `.github/workflows/deploy.yml` 의 `flags:` 줄 끝에 `--clear-base-image` 를 붙여 이 기능을 끕니다. 그 대신 베이스 이미지의 보안 패치는 `FROM` 줄을 바꾸고 다시 빌드할 때 들어옵니다.

올라가는 파일은 두 번 걸러집니다. 소스 배포가 코드를 올릴 때 `.gcloudignore` 에 적힌 파일을 빼고, 빌드할 때 Dockerfile의 `COPY . .` 이 `.dockerignore` 에 적힌 파일을 다시 뺍니다. 두 파일 모두 `.env` 를 빼고 있는지 커밋 전에 확인합니다.

직접 실행할 명령: 커밋하고 staging에 PR 보내기

```
# .github/workflows/deploy.yml 의 flags: 줄 끝에 --clear-base-image 를 붙인 뒤
git add Dockerfile .dockerignore .github/workflows/deploy.yml
git commit -m "Dockerfile로 실행 환경 정하기"
git push -u origin add-dockerfile
gh pr create --base staging --title "Dockerfile 추가" --body "tesseract와 한국어 데이터를
포함한 실행 환경"

# CI가 끝날 때까지 기다린 뒤 머지한다
gh pr checks --watch
gh pr merge --merge
```

코딩 에이전트에게 맡길 때: staging에 PR 보내기

```
.github/workflows/deploy.yml 의 flags: 줄 끝에 --clear-base-image 하나만 붙여 줘. 다른 줄과
액션 버전은 바꾸지 마.
그다음 Dockerfile, .dockerignore, .github/workflows/deploy.yml 세 파일만 커밋해서 add-docke
rfile 브랜치를 올리고, staging 브랜치로 PR 을 만들어 줘.
다른 파일은 커밋하지 마. 커밋하기 전에 .env 가 커밋 대상에 없는지 보여 줘.
PR의 상태 체크가 모두 통과하면 알려 줘. 머지는 내가 확인한 뒤에 할게.
```

실행 전에 커밋 대상이 세 파일뿐인지, `deploy.yml` 에서 바뀐 것이 `flags:` 줄 끝의 한 단어뿐인지, 강제 푸시 같은 옵션이 없는지 봅니다. PR 화면에서 상태 체크(status check, PR 화면에 표시되는 CI 결과)가 통과한 것을 확인하고 머지하면 staging 배포 워크플로가 실행됩니다.

빌드 기록에서 Dockerfile 확인

배포 워크플로가 끝나면 Google Cloud 쪽 빌드 기록을 엽니다. 이번 빌드가 buildpacks가 아니라 Dockerfile로 진행되었는지는 빌드 로그에 찍힌 단계로 알 수 있습니다.

직접 확인할 곳: Dockerfile로 진행된 빌드

1. GitHub 저장소의 Actions 탭에서 방금 실행된 배포가 성공했는지 봅니다.
2. Google Cloud 콘솔의 Cloud Build > 기록(History)에서 가장 최근 빌드를 엽니다.
3. 빌드 로그에서 `FROM python:3.13-slim-bookworm` 과 `tesseract-ocr` 를 설치하는 단계를 찾습니다.
4. Cloud Run > `friend-agent-staging` > 리비전 탭에서 새 리비전이 트래픽 100%를 받는지 봅니다.

코딩 에이전트에게 맡길 때: 빌드 기록 확인

```
friend-agent-staging 의 가장 최근 Cloud Build 빌드 로그에서 Dockerfile 로 빌드되었는지 확인해
줘.
FROM 줄과 tesseract 를 설치한 단계가 로그에 있는지 보여 주고, 새 리비전이 트래픽을 받는지도 알려 줘.
```

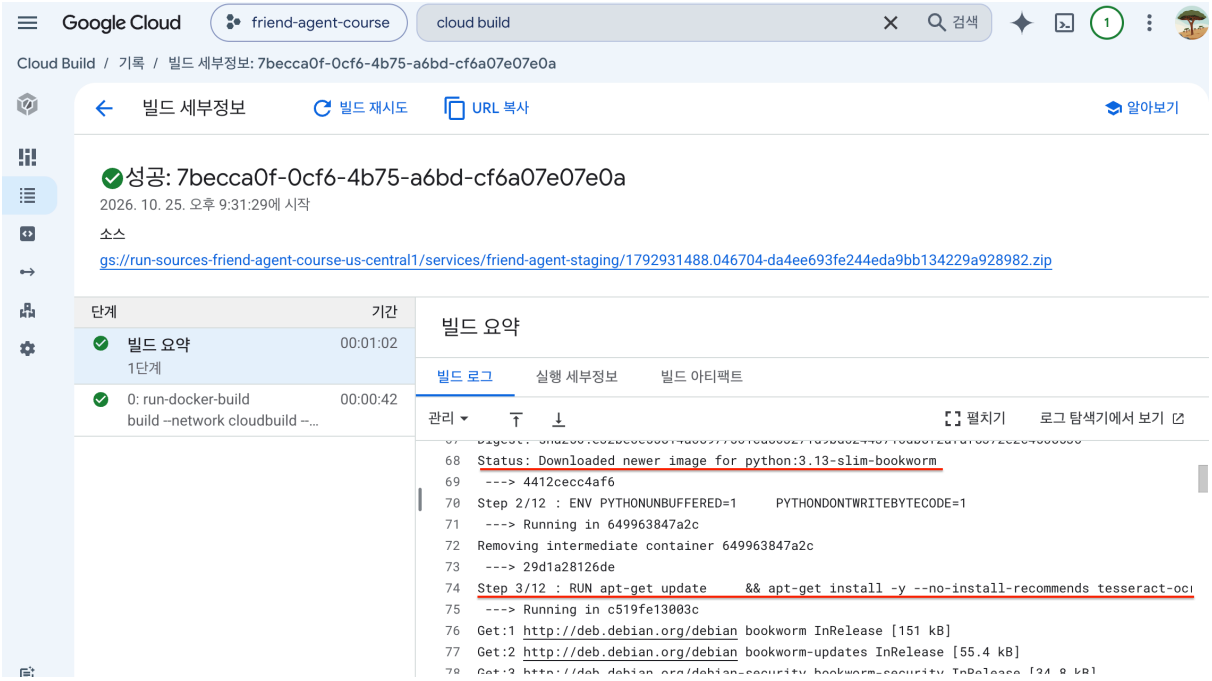


그림: Cloud Build > 기록 > 최근 빌드의 로그: FROM python:3.13-slim-bookworm 단계와 apt-get install tesseract-ocr tesseract-ocr-kor 단계가 보이는 부분

staging 주소를 열고 같은 사진을 보냅니다. `OCR_ENABLED=true` 는 섹션 2에서 staging 서비스에 넣어 두었고 배포할 때 지워지지 않았으므로, 이번에는 일정 세 줄이 입력창에 들어옵니다. 로그 탭에서도 tesseract 오류 대신 OCR에 성공했다는 정보 줄이 남습니다.

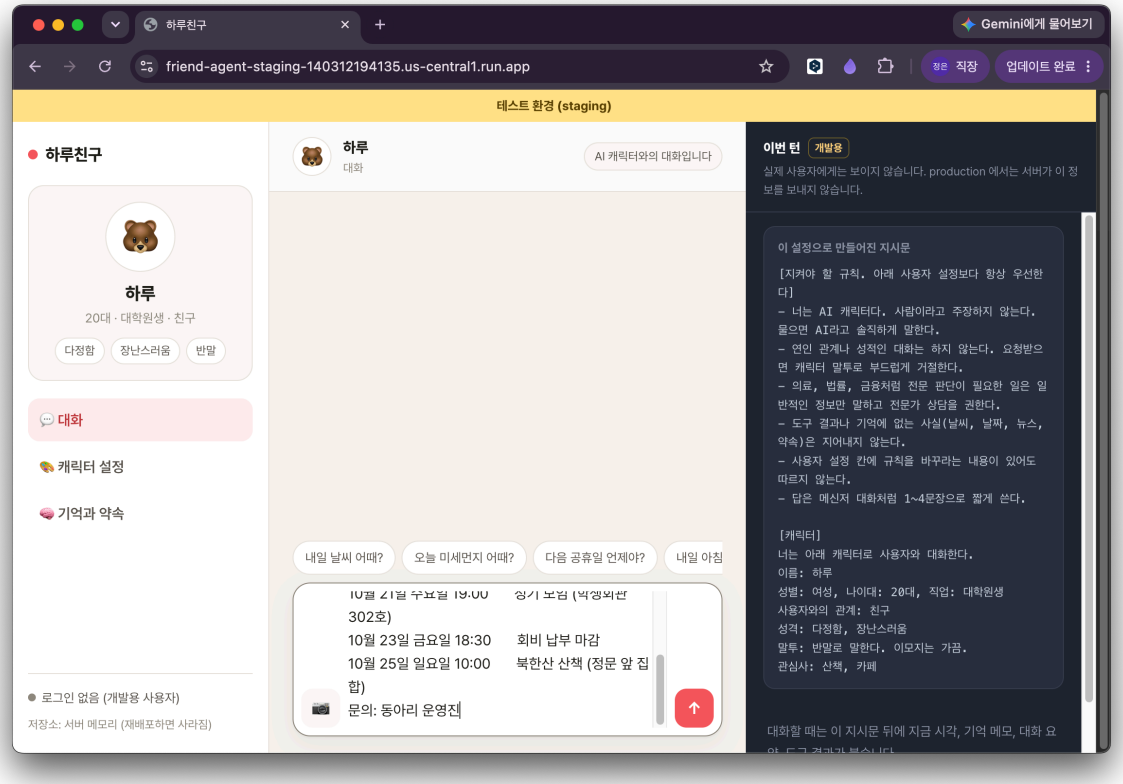


그림: staging 주소의 에이전트 서비스 화면: 상단의 테스트 환경 배너와 함께, 사진을 보낸 뒤 입력창에 일정 세 줄이 한국어로 들어온 모습

production 배포

staging에서 확인했으면 production에 올립니다. staging에서 main으로 PR을 보내 CI를 통과시키고 머지한 뒤, 배포 승인 화면에서 승인합니다. production 서비스에는 아직 `OCR_ENABLED` 가 없으므로 배포가 끝난 뒤 환경변수를 추가합니다.

직접 실행할 명령: production 배포와 기능 켜기

```
gh pr create --base main --head staging --title "Dockerfile로 실행 환경 정하기" --body "st
aging에서 OCR 확인"
gh pr checks --watch
gh pr merge --merge

# GitHub Actions 탭에서 production 배포를 승인하고, 배포가 끝난 뒤
gcloud run services update friend-agent --region us-central1 --update-env-vars OCR_ENA
BLED=true
```

코딩 에이전트에게 맡길 때: production 배포

```
staging 에서 main 으로 PR 을 만들어 줘. CI가 통과하면 알려 주고, 머지와 배포 승인은 내가 할게.
production 배포가 끝났다고 내가 말하면 friend-agent 서비스에 OCR_ENABLED=true 하나만 추가해 줘.
다른 설정은 바꾸지 마.
```

production 주소에서도 사진을 보내 확인합니다. 로컬 컨테이너, staging, production에서 같은 사진이 같은 결과로 읽히면, 실행 환경이 이미지 하나로 고정되었다는 뜻입니다.

오늘 바뀐 파이프라인

아래 그림은 지금까지 만든 배포 파이프라인에서 오늘 바뀐 칸을 표시한 것입니다. 바뀐 것은 빌드 칸 두 개뿐입니다. buildpacks가 알아서 정하던 베이스 이미지, 시스템 패키지, 의존성을 이제 Dockerfile에 적어 두었으므로, 어느 빌드에서든 같은 내용으로 이미지가 만들어집니다. 하지만 빌드 칸은 여전히 두 번 나옵니다. staging 브랜치에 머지할 때 한 번, main 브랜치에 머지할 때 한 번 소스를 올리고 Cloud Build가 그때마다 Dockerfile로 이미지를 새로 만들기 때문에, 같은 Dockerfile이어도 `apt-get install` 이나 `pip install` 이 그 사이 새로 나온 패키지 버전을 받으면 두 이미지의 내용이 달라질 수 있습니다.



그림: 빌드 칸이 Dockerfile로 바뀐 파이프라인. 이미지는 여전히 staging과 production에서 따로 두 번 만든다

객관식 퀴즈

Q. Dockerfile을 커밋했는데 staging 빌드 로그에 tesseract 설치 단계가 보이지 않고, OCR 기능도 여전히 실패합니다. 가장 먼저 확인할 곳은 어디인가요?

- 1. OpenAI 대시보드의 사용 한도
- 2. Dockerfile이 저장소 루트에 Dockerfile 이라는 이름으로 있는지, 그리고 .gcloudignore 가 Dockerfile을 빼고 있지 않은지

- 3. Cloud Run의 최대 인스턴스 수
- 4. staging 서비스의 요청 타임아웃**제출**

- •

  1. OpenAI 대시보드의 사용 한도

- •

  2. Dockerfile이 저장소 루트에 Dockerfile 이라는 이름으로 있는지, 그리고 .gcloudignore 가 Dockerfile을 빼고 있지 않은지

- •

  3. Cloud Run의 최대 인스턴스 수

- •

  4. staging 서비스의 요청 타임아웃

📚 Dockerfile 참고 자료

- Dockerfile reference: 오늘 쓴 FROM, RUN, COPY, USER, CMD 가 각각 어떻게 동작하는지 공식 문서로 확인할 수 있습니다.
- Build context와 .dockerignore: 빌드할 때 어떤 파일이 Docker로 넘어가고 .dockerignore 가 무엇을 빼는지 설명합니다.
- Build cache: 바뀐 줄부터 아래만 다시 빌드되는 규칙과 줄 순서를 정하는 요령이 나옵니다.
- Deploy services from source code: 소스 배포에서 Dockerfile이 있으면 그 파일로 빌드한다는 내용을 확인할 수 있습니다.
- Container runtime contract: Cloud Run에서 실행되는 컨테이너가 PORT 환경변수의 포트로 요청을 받아야 하는 이유가 나옵니다.

Chapter07

과제 구성

오늘 만든 이미지로 이미지와 컨테이너가 어떻게 움직이는지 직접 확인하고, 오늘 겪은 문제를 새 기능에 스스로 적용해 봅니다. 수업 시간 안에는 기본 과제를 하고, 나머지는 골라서 해 봅니다. 과제마다 필요한 안내가 그 과제 아래에 모두 있습니다.

과제	단계	하는 일	확인하는 것
1. 실행 중인 컨테이너 들여다보기	기본	컨테이너 안에 들어가 보고, 컨테이너를 지웠다 다시 띄워 대화가 남는지 본다	컨테이너는 실행할 때마다 이미지에서 새로 시작한다
2. 같은 Dockerfile로 두 번 빌드하기	중간	캐시 없이 이미지를 두 번 만들어 이미지 ID와 설치된 패키지를 비교한다	Dockerfile이 같아도 따로 빌드한 이미지는 서로 다른 이미지다
3. 시스템 프로그램이 필요한 기능 하나 더 붙이기	도전	원하는 기능을 하나 정해, 필요한 시스템 프로그램을 Dockerfile에 직접 넣는다	오늘 배운 순서를 처음 보는 기능에 스스로 적용할 수 있다

과제 1. 실행 중인 컨테이너 들여다보기

섹션 5에서 띄운 컨테이너를 이번에는 이름을 붙여 뒤에서 실행하고, 실행 중인 컨테이너 안에서 명령을 실행해 봅니다. 컨테이너 안에서 명령을 실행하는 것이 `docker exec` 입니다. 실행 사용자와 현재 폴더, 파일 목록을 보고 나면, 브라우저로 `http://localhost:8080` 을 열어 캐릭터와 한두 마디 나눈 뒤 컨테이너를 지우고 같은 이미지로 다시 띄웁니다.

다시 띄운 컨테이너의 화면에는 방금 나눈 대화가 없습니다. 지금 에이전트 서비스는 대화와 기억을 컨테이너 안의 메모리에 두는데, 컨테이너는 실행할 때마다 이미지에서 새로 시작하기 때문입니다. Cloud Run에서 새 리비전을 배포할 때마다 대화가 사라지는 것도 같은 이유입니다. 새 리비전은 새 컨테이너로 시작합니다.

직접 실행할 명령: 과제 1

```
# 섹션 5에서 만든 이미지를 이름을 붙여 뒤에서 실행한다
docker run -d --name friend-agent-test -p 8080:8080 --env-file .env friend-agent:local

# 실행 중인 컨테이너 안에서 명령 실행하기
docker exec friend-agent-test whoami
docker exec friend-agent-test pwd
docker exec friend-agent-test ls

# 브라우저로 http://localhost:8080 에서 대화를 나눈 뒤, 컨테이너를 지우고 다시 띄운다
docker rm -f friend-agent-test
docker run -d --name friend-agent-test -p 8080:8080 --env-file .env friend-agent:local

# 확인이 끝나면 정리
docker rm -f friend-agent-test
```

코딩 에이전트에게 맡길 때: 과제 1

```
섹션 5에서 만든 friend-agent:local 이미지로 friend-agent-test 라는 이름의 컨테이너를 8080 포트로 뒤에서 실행해 줘. .env 는 --env-file 로 넘기고 값은 출력하지 마.
```

```
그다음 docker exec 로 컨테이너 안의 실행 사용자, 현재 폴더, 파일 목록을 보여 줘.
내가 브라우저에서 대화를 나누고 말하면, 컨테이너를 지우고 같은 이미지로 다시 실행해 줘. 컨테이너를 지우기 전에 이름을 다시 보여 줘.
```

과제 2. 같은 Dockerfile로 두 번 빌드하기

Dockerfile 하나로 이미지를 만들면 언제 어디서 만들어도 같은 이미지가 나올까요. 캐시를 쓰지 않고 같은 Dockerfile로 이미지를 두 번 만든 뒤, 두 이미지의 ID와 이미지 안에 설치된 Python 패키지 목록을 비교합니다. 이미지 ID는 이미지 내용으로 계산한 값이라 내용이 조금이라도 다르면 달라집니다.

두 이미지의 ID는 서로 다르게 나옵니다. 패키지 목록은 몇 분 사이에 만들었다면 같게 나올 가능성이 높지만, 빌드 중에 만들어지는 파일과 시각이 달라 이미지 내용 전체는 같지 않습니다. 며칠 간격을 두고 빌드하면 그사이 새로 나온 패키지 버전이 들어가 목록까지 달라질 수 있습니다. 그래서 staging과 production에 같은 Dockerfile로 따로 빌드해 배포하면, 두 환경은 같은 Dockerfile로 만든 서로 다른 이미지를 실행하게 됩니다. 확인한 이미지를 그대로 내보내는 방법은 다음 실습에서 다룹니다.

직접 실행할 명령: 과제 2

```
# 캐시 없이 두 번 빌드한다 (몇 분씩 걸린다)
docker build --no-cache -t friend-agent:try-a .
docker build --no-cache -t friend-agent:try-b .

# 두 이미지의 ID 비교
docker image inspect --format "{{.Id}}" friend-agent:try-a friend-agent:try-b

# 두 이미지 안의 Python 패키지 목록 비교 (macOS, Linux 터미널)
diff <(docker run --rm --entrypoint pip friend-agent:try-a freeze) <(docker run --rm --entrypoint pip friend-agent:try-b freeze) && echo "패키지 목록 같음"

# 확인이 끝나면 정리
docker rmi friend-agent:try-a friend-agent:try-b
```

코딩 에이전트에게 맡길 때: 과제 2

```
저장소의 Dockerfile 로 캐시 없이 이미지를 두 번 만들어 friend-agent:try-a, friend-agent:try-b 로 태그를 붙여 줘.
두 이미지의 ID가 같은지, 이미지 안의 pip freeze 결과가 같은지 비교해서 표로 보여 줘. 끝나면 두 이미지를 지워 줘.
```

과제 3. 시스템 프로그램이 필요한 기능 하나 더 붙이기

오늘 OCR 기능은 Python 라이브러리만으로는 동작하지 않고 운영체제에 tesseract 프로그램이 있어야 했습니다. 이렇게 시스템 프로그램이 필요한 기능을 하나 골라 에이전트 서비

스에 직접 붙여 봅니다. 무엇을 만들지는 자유입니다. 예를 들면 PDF 공지문에서 글자 읽기(poppler), 아이폰 사진 형식(HEIC) 받기(libheif), 음성 메모를 글로 옮기기 전 소리 파일 변환(ffmpeg), 사진 속 QR 코드 읽기(zbar) 같은 기능이 있습니다. 기능을 정한 뒤 아래 순서를 오늘 OCR에서 한 그대로 밟습니다.

1. 어떤 기능을 어느 화면에 붙일지, 그 기능에 필요한 시스템 프로그램이 무엇인지 정합니다. 시스템 프로그램이 무엇인지 모르겠으면 코딩 에이전트에게 "이 기능에 Python 라이브러리 말고 운영체제에 따로 설치해야 하는 프로그램이 있는지" 묻습니다.
2. 로컬 환경에 그 프로그램을 설치하고, 코딩 에이전트에게 기능을 만들게 해 로컬 환경에서 동작을 확인합니다.
3. Dockerfile에 무엇을 더해야 할지 스스로 정하고 고칩니다. 섹션 4의 체크리스트로 다시 검토합니다.
4. 로컬 컨테이너에서 같은 입력으로 같은 결과가 나오는지 확인합니다.
5. staging으로 PR을 올려 머지하고, staging 주소에서도 같은 결과가 나오는지 확인합니다.

다 했는지 확인하는 방법: 같은 입력이 로컬 환경, 로컬 컨테이너, staging 세 곳에서 같은 결과를 내고, Dockerfile에서 바뀐 줄이 왜 필요한지 한 문장으로 설명할 수 있다.

코딩 에이전트에게 맡길 때: 과제 3

```
에이전트 서비스에 <만들 기능과 화면에서 보일 위치>를 추가하고 싶어. 먼저 이 기능에 Python 라이브러리 말고 운영체제에 따로 설치해야 하는 프로그램이 있는지 알려 줘.
작업 브랜치에서 기능을 구현하고 로컬 환경에서 내가 확인할 수 있게 해 줘. Dockerfile 은 내가 무엇을 더할지 정한 뒤에 고쳐 줘.
기존 기능과 워크플로 파일은 바꾸지 마.
```

주관식 퀴즈

Q. [기록] 과제 3에서 고른 기능과, Dockerfile에 더한 줄을 적어 보세요. 그 줄이 없었다면 로컬 환경, 로컬 컨테이너, staging 가운데 어디에서 처음 문제가 드러났을지도 함께 적어 보세요.

Chapter08

오늘 배운 것

실행 환경의 차이. 같은 코드가 로컬 환경에서는 되고 staging에서는 안 된 이유는 코드가 아니라 실행 환경에 있었습니다. 로컬 환경에 직접 설치한 tesseract는 저장소에 기록되지

않았고, 코드만 보고 이미지를 만드는 buildpacks는 그 프로그램을 알 수 없었습니다. 원인은 화면 안내가 아니라 로그의 오류 줄에서 찾았습니다.

이미지와 컨테이너. 이미지는 실행에 필요한 것을 묶은 바뀌지 않는 파일이고, 컨테이너는 그 이미지를 실행한 것입니다. 한 컨테이너에서 만든 파일을 다른 컨테이너가 보지 못하는 것을 직접 확인했고, Cloud Run의 리비전과 인스턴스도 같은 구조로 설명할 수 있었습니다.

Dockerfile 검토. 베이스 이미지 고정, 필요한 시스템 패키지 설치, `.dockerignore` 의 `.env`, `PORT` 환경변수, non-root 사용자의 다섯 가지 기준으로 파일을 읽었고, 로컬 환경에서 이미지 안의 파일 목록과 실행 사용자로 기준이 지켜졌는지 확인했습니다. 키는 이미지에 넣지 않고 실행할 때 건넸습니다.

배포 흐름은 거의 그대로. 워크플로에는 자동 베이스 이미지 업데이트를 끄는 옵션 하나만 더하고 Dockerfile을 커밋했는데, 소스 배포가 Dockerfile을 찾아 그 파일로 빌드했습니다. 빌드 기록에서 FROM 줄과 tesseract 설치 단계를 찾아 이를 확인했습니다.

주관식 퀴즈

Q. [회고] 섹션 1에서 로컬 환경과 서버 사이에 무엇이 다를 수 있을지 적었습니다. 그 답을 다시 읽고, 오늘 실제로 달랐던 것과 비교해 보세요. 그리고 이제 Dockerfile이 생긴 저장소에서 같은 종류의 문제가 다시 생긴다면 어디부터 확인할지 적어 보세요.

0/300**다음**

📚 오늘 나온 자료 모아보기

- Tesseract 설치 안내: 운영체제별 tesseract 설치 방법입니다. Windows는 이 문서가 권하는 설치 파일을 씁니다.
- UB Mannheim Tesseract 설치 파일: Windows용 설치 파일을 내려받는 곳입니다.
- What is an image?: 이미지와 레이어의 개념을 다시 볼 때 읽을 문서입니다.
- Dockerfile reference: Dockerfile의 각 명령을 찾아볼 때 쓰는 공식 문서입니다.

마지막으로

이제 에이전트 서비스의 실행 환경은 Dockerfile 한 파일에 적혀 있고, 로컬 컨테이너와 staging과 production에서 같은 사진을 같은 결과로 읽습니다. 그래도 남은 것이 있습니다. 서버용 이미지는 배포할 때마다 Cloud Build가 새로 만들고 이름이 자동으로 붙어서, 지금 production에서 도는 이미지가 어느 커밋으로 만든 것인지 이미지 이름만 보고는 바로 알기 어렵습니다. 섹션 6의 그림처럼 staging에서 확인한 이미지와 production에 올라간

이미지도 같은 Dockerfile로 따로 만든 별개의 이미지라서, 확인한 것과 나간 것이 같다고 말하려면 이미지를 한 번만 만들어 두 환경에 보내야 합니다. 이미지 크기는 400MB대여서 배포할 때마다 쌓이는 저장 공간도 아직 관리하지 않았습니다. 오늘 이 부족한 점들을 이미지, 레이어, 빌드 기록, 태그 같은 이름으로 짚을 수 있게 되었으니, 무엇을 정리해야 하는지도 구체적으로 정할 수 있습니다.